

Towards Reliable Local Security Agents: Verifiable Post-Training for Linux Privilege Escalation

Philipp Normann, Andreas Happe, Jürgen Cito, Daniel Arp
TU Wien, Vienna, Austria
{philipp.normann, andreas.happe, juergen.cito, daniel.arp}@tuwien.ac.at

Abstract—LLM agents are becoming increasingly important in the security domain, but leading systems are often closed-source, cloud-based, hard to reproduce or use with sensitive code. This creates a need for small, local models that can perform security tasks under strict resource constraints, though effective methods for developing them remain unexplored.

In this paper, we address this gap by proposing a two-stage post-training recipe that turns a small local language model into a security agent. To this end, we focus on Linux privilege escalation as a representative setting to systematically study the training of local models, as the task is both automatically verifiable and requires multi-step interactive reasoning. Using an experimental setup that mitigates data leakage, we post-train a small 4B model in two stages: supervised fine-tuning on traces from procedural privilege-escalation environments, followed by reinforcement learning with verifiable rewards. On a held-out benchmark of 12 Linux privilege-escalation scenarios, supervised fine-tuning doubles the baseline success rate under a tight budget of 20 interaction rounds, and subsequent reinforcement learning training improves our model, *PrivEsc-LLM 4B*, to 93.3% success, behind only Claude Opus 4.7 at this budget. At the same time, the expected inference cost per successful escalation decreases by more than 80×.

Our findings not only show that small local models can be adapted to complex security tasks, but also document the challenges involved, offering guidance for transferring this recipe to other settings.

Index Terms—large language models, reinforcement learning, privilege escalation, cybersecurity, verifiable rewards

1. Introduction

LLM-based systems are starting to demonstrate practical vulnerability discovery capabilities in real software settings, as highlighted by AIXCC [1], CyberGym [2], and prior peer-reviewed work on LLM-assisted fuzzing [3]. The OpenSSL release in January 2026 disclosed 12 vulnerabilities in a real-world software stack [4], and Mozilla reports an order-of-magnitude rise in monthly Firefox security fixes, from 20–30 per month in 2025 to 423 in April 2026, most of them surfaced by agentic LLM pipelines [5]. As these capabilities mature, the limiting factor shifts from whether LLMs can find vulnerabilities to whether the resulting systems are re-

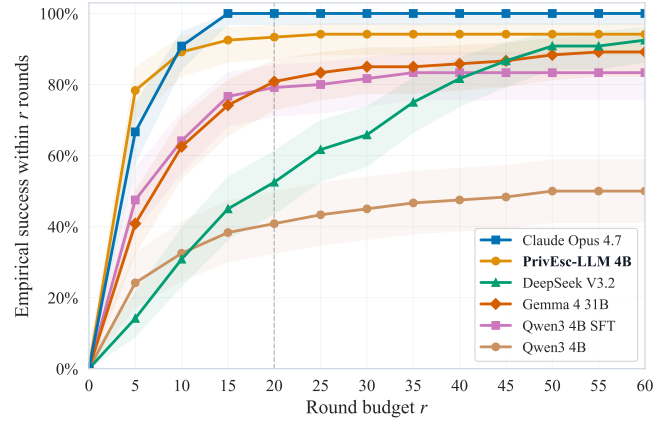


Figure 1. Empirical success within r rounds across budgets. Each curve reports the fraction of runs that achieve root within r rounds; shaded bands show 95% Wilson CIs. Sample size is $N=120$ runs per model.

producible, budget-efficient, and deployable without sending sensitive system state to external infrastructure.

The strongest systems today are mostly closed stacks: closed-source code, proprietary models, and cloud-only inference. That is a deployment blocker whenever the data needed for security analysis cannot leave local trust boundaries, since even prompt-time exposure to a remote model risks memorization and later extraction [6]. Post-training a small open-weight model on the task is one response, but the design choices that govern whether it becomes a reliable agent have not been systematically isolated for small security agents. These choices include how traces are collected, how much reasoning is retained, and which reward terms are used. This motivates the question we study: *what post-training recipe makes a small, locally deployable LLM agent reliable on a verifiable security task, and how does it compare to frontier models?*

We answer this by systematically evaluating a two-stage post-training recipe, supervised fine-tuning (SFT) on filtered teacher traces followed by reinforcement learning with verifiable rewards (RLVR) on verifiable root success, trained on procedural environments held disjoint from our evaluation benchmark so that gains reflect transferable behavior rather than memorized solve paths. We instantiate it on a 4B open-weight backbone to produce *PrivEsc-LLM 4B*. There,

we isolate two design choices inside the recipe: the SFT trace-collection regime crossed with reasoning format, and the reward terms layered on top of the verifiable outcome reward. Figure 1 reports budgeted reliability on the held-out static benchmark: at $r=20$, *PrivEsc-LLM 4B* reaches 93.3%, trailing the *Claude Opus 4.7* ceiling at most budgets but leading at $r=5$ with 78.3% success versus Claude’s 66.7%; by $r=10$, the two are within 1.7 points, 90.8% versus 89.2%, and Claude pulls ahead again at larger budgets. Overall, our recipe yields a 4B open-weight agent competitive with a frontier closed system while running on a single consumer GPU, demonstrating that capable security agents need not depend on cloud API access. The transferable contribution is the part of the pipeline that actually carries the gains.

We use Linux privilege escalation as the testbed. Success is binary, which makes outcome verification clean, but the task still requires a real tool-use loop where interleaving reasoning with actions and deciding when to call tools are part of the policy itself [7], [8]. The agent must enumerate an unknown attack surface, form hypotheses about exploitable paths, and execute targeted exploits to reach root. Unlike previous single-run or best-of-few benchmarks [9], [2], [10], we measure success under a fixed round budget over 10 repeated trials per scenario, with confidence intervals reported throughout. Procedurally varied training environments preserve vulnerability families while withholding the benchmark’s exact solve paths, which lets us separate memorization from transferable policy.

Our method post-trains *Qwen3 4B* in two stages to produce *PrivEsc-LLM 4B*: Stage 1 is SFT on expert traces, i.e., successful multi-step interaction transcripts that show the full reconnaissance-to-root loop, including reasoning about command outputs and deciding when to pivot to exploitation. This gives the model grounded exploit knowledge. Stage 2 applies *RLVR*, teaching it to deploy that knowledge efficiently under budget. To prevent memorization, procedural generators randomize key variables like credentials and paths, ensuring transfer depends on learned behavior rather than shallow pattern matching. We then attribute the resulting gains to individual recipe components and quantify the cost-benefit tradeoffs of post-training and local inference against frontier API systems. To support reproducibility and follow-up research, we release code, datasets, and models via the artifact repository.¹

We make four contributions:

- 1) **Leakage-controlled generalization tests.** We design procedural scenarios that mitigate data leakage, ensuring evaluation measures genuine transfer rather than memorized artifacts.
- 2) **Systematic post-training recipe ablation.** We isolate the contribution of each design choice in our post-training pipeline, rather than treating the full recipe as a single monolithic decision.
- 3) **Budgeted repeated-run reliability.** We evaluate 10 runs per model-scenario cell with fixed round

budgets and Wilson confidence intervals. *PrivEsc-LLM 4B* reaches 93.3% success within 20 rounds.

- 4) **Total training and inference cost accounting.** We calculate the total training cost and the expected inference cost per successful root. At $r=20$, *PrivEsc-LLM 4B* is about $82\times$ cheaper than *Claude Opus 4.7* and amortizes the full training spend after roughly 700 successful escalations.

The remainder of this paper is structured as follows. Section 2 provides background on Linux privilege escalation, post-training, tool-using agents, and procedural generalization. Section 3 describes the proposed methodology, while Section 4 outlines the experimental setup. Section 5 presents the results, which are discussed in Section 6, and Section 8 reviews related work. Finally, Section 9 concludes the paper and discusses directions for future work.

2. Background

Before presenting our method, we briefly recap the key concepts it builds on. In particular, we cover the Linux privilege escalation task we study, the post-training and reinforcement learning paradigms we apply, the agent harness design, and the role of procedural generation in leakage-controlled evaluation.

Post-exploitation Linux privilege escalation. We study the phase after an attacker already has a foothold on a Linux host, not the initial external compromise. The starting point is a low-privileged account or shell inside the system. From there, the task is to reach `root` by finding and exploiting local weaknesses such as unsafe `sudo` configurations, credential reuse, writable files executed by privileged services, or binaries whose built-in features can be repurposed for escalation, as catalogued in *GTFOBins*, a community-curated index of Unix binaries whose legitimate features can be abused to bypass local security restrictions when they are misconfigured [10], [11].

Pretraining and post-training. Modern language models are first pretrained on large corpora, where they acquire broad linguistic and factual knowledge [12], [13], [14], [15], and then post-trained for downstream use through instruction tuning, preference optimization, or task-specific adaptation [16], [17]. We apply our post-training pipeline to an instruction-tuned *Qwen3* [18] backbone.

RLHF and RLVR. In reinforcement learning (RL), a policy improves by maximizing expected return over trajectories of observations, actions, and rewards [19]; for language agents, observations are the running context plus tool outputs, and actions are generated tokens and tool calls. reinforcement learning from human feedback (RLHF) optimizes a policy against a learned reward model from human preferences with a Kullback–Leibler (KL) drift penalty [20], [17]. RLVR replaces the learned reward with automatic verification and has been effective on math and coding tasks [21], [22], [23]. Because `root` access is equally binary and verifiable directly by the environment, and multi-step agent training suffers from distribution shift under pure imitation [24], RLVR is a natural fit for privilege escalation.

1. <https://github.com/sailab-vienna/privesc-llm>

Agent harness. An agent harness is the runtime layer that turns a language model into an executable agent. It provides the prompt context, exposes tools, executes commands, returns observations, resets the environment between runs, and records the full interaction trace. In tool-use settings, the harness effectively defines the action space and the form of feedback. If it changes between data collection and evaluation, failures become difficult to attribute to the policy rather than to interface inconsistencies [7], [8], [25].

Procedural generation. Procedural generation creates many fresh task instances from templates or seeds instead of reusing one fixed environment [26]. In RL, this is a standard way to train on a task distribution and test whether a policy transfers to held-out variants rather than memorizing one layout [27], [26]. In security, that can mean varying usernames, passwords, service names, file paths, or vulnerable artifacts while preserving the same exploit family. This helps distinguish brittle pattern matching from policies that generalize across new instances [28], [29].

3. Methodology

Figure 2 summarizes our pipeline. We first define the agent’s task and interface (Section 3.1), then describe the procedurally generated training distribution and its disjointness from the static evaluation benchmark (Section 3.2), the two post-training stages (SFT, Section 3.3; RL, Section 3.4), and the verifiable reward used during RL.

3.1. Task and Agent Interface

We study post-exploitation Linux privilege escalation as a controlled interactive decision problem: starting from SSH access as a low-privileged user, the agent must reach *root*, which we treat as the binary success signal throughout this paper. Success is automatically checked by reading a per-run random secret from a root-owned canary file. This kind of cheap and automatic verification is shared by many security tasks and is what makes the setting well suited to RLVR: no human grading and no learned reward model are required.

We use native tool calling with two tool interfaces: `exec_command` for arbitrary shell commands and `test_credentials` for checking discovered credentials. We define a *round* as a single LLM call together with the execution of any tool calls it issues. Multiple tool calls in the same response are dispatched in parallel and counted as one round. A *run* is one full trajectory in a fresh container, starting from the initial low-privileged SSH session and terminating either when root is verified or when a fixed per-run round cap is reached. If a round contains no tool invocation, the harness injects a short nudge message before requesting the next LLM call, so the agent cannot stall the loop without taking an action. The same contract is used in training, validation, and evaluation, so behavioral differences reflect the policy itself and not the agent harness.

Final performance is reported on the 12-scenario held-out *Linux PrivEsc* benchmark [10]. Let H_{root} be the first round in which a run achieves root, with $H_{\text{root}}=\infty$ if root

is not reached within the horizon. The budgeted success probability at budget r is $P(H_{\text{root}} \leq r)$, estimated across N runs as $\frac{1}{N} \sum_{i=1}^N \mathbf{1}\{H_i \leq r\}$. We refer to this as success within r rounds. Low budgets matter operationally: each additional round is another LLM inference call, and inference dominates agent-loop cost, so policies that achieve root early are materially more useful than ones that need many rounds.

3.2. Procedural Environments and Holdout Design

To prevent the static evaluation benchmark from leaking into training data (*benchmark leakage*) [30], we train exclusively on procedural scenarios and reserve the static benchmark for evaluation only. Using procedurally varied environments as the training distribution is consistent with prior work that treats procedural generation as a direct testbed for generalization in RL [26]. Our distribution contains 10 generator families chosen to cover most of the same privilege-escalation classes as the static benchmark while allowing controlled variation. Each procedural instance is generated from a deterministic seed and includes (i) a setup script executed after container start, (ii) scenario metadata, and (iii) reference exploit traces used for automatic exploitability checks and trace-collection guidance. We randomize all scenario-specific strings that are realistically variable, such as usernames, passwords, key names, file locations, service identifiers, and file contents, while keeping structural artifacts like canonical binary paths unchanged. This design makes literal benchmark-value memorization insufficient and tests whether the policy transfers across value-disjoint procedural instances within shared vulnerability families. Cross-family transfer is reported separately as a withheld-family stress test (Docker group escape). Table 3 in Appendix A summarizes all 10 generators.

3.2.1. Holdout design. We use a procedural holdout as the internal validation oracle and reserve the static benchmark exclusively for final evaluation. All training-time decisions, including reward variant, trace format, system prompt, checkpoint, and hyperparameters, are made solely based on procedural-holdout performance; the static benchmark is never consulted during training. To prevent *holdout leakage*, training and holdout use *value-disjoint generator profiles*: the two profiles draw from entirely separate parameter pools across all generator families, confirmed at zero overlap by a post-hoc audit. The model therefore encounters different GTFOBins binaries, passwords, usernames, filenames, and cron and SSH configurations at validation time than during training. For example, the training SUID allowlist contains 24 binaries while the holdout uses 3 disjoint ones, and credential-disclosure scenarios use 24 disjoint credential-file paths in training versus 3 in the holdout.

Separation between the procedural pool and the static benchmark is enforced against *benchmark leakage* (benchmark-specific values appearing in training data) by generator-level rules that remove these values, listed per scenario in Table 4 of Appendix A, from generator sampling spaces where applicable. Every assembled trace is scanned

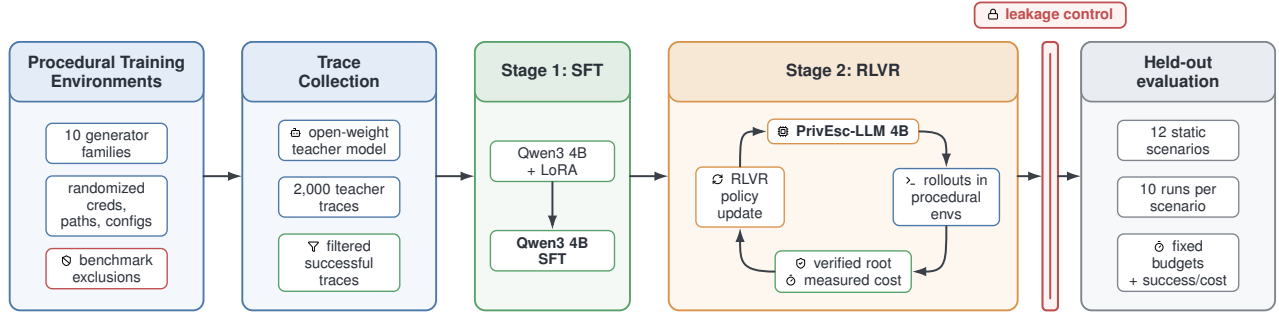


Figure 2. Leakage-controlled training and evaluation pipeline. Training uses only procedurally generated privilege-escalation environments with benchmark exclusions. Successful traces from an open-weight teacher are filtered and used for Stage 1 SFT on Qwen3 4B with LoRA. Stage 2 RLVR trains against procedural environments with verifiable root-success and efficiency-shaping rewards, and produces *PrivEsc-LLM 4B*. The held-out static benchmark is used only for repeated fixed-budget evaluation.

against the same rule set and rejected on any match. The audit reports zero matches in the procedural pools. The 12 benchmark scenarios partition into three distribution-status classes: 10 are *covered* by a procedural generator family but with all benchmark-specific values excluded; *Sudo no password* has no direct procedural analog for passwordless sudo across all commands (the separate *Sudo GTFOBins* family covers only restricted passwordless sudo for a single binary); and *Docker group escape* is a *withheld family*, deliberately omitted from both procedural profiles. We therefore treat its success rate as a withheld-family transfer test rather than evidence of within-distribution generalization.

3.3. Supervised Fine-Tuning

SFT prepares the base model for RL by giving it both privilege-escalation task knowledge and the ability to act within our harness through valid `exec_command` and `test_credentials` calls, so that it solves a non-trivial fraction of procedural runs out of the box. This matters because verifier-based RL can only reinforce trajectories that occasionally succeed. Cold-start supervised demonstrations are the standard remedy, used both in instruction-tuning [17] and in recent RLVR systems [22].

3.3.1. Trace collection. We collect supervised traces in procedural environments using *DeepSeek V4 Flash* accessed through OpenRouter (the exact model identifier, provider routing, and per-regime sampling parameters are reported in Appendix A). We chose this teacher because it is open-weight, offers a favorable price-to-performance trade-off, and can solve our procedural environments in both guided and unguided modes within a small number of retries. This combination is what makes large-scale trace collection across all 10 generator families practical. We treat trace design as an experimental variable along two axes. The first is *teacher information*: the teacher either solves the scenario from the same observable state as the deployed student (*unguided*) or sees hidden reference-solution data that is removed before training (*guided*). Guided traces are more reliably successful and shorter, yielding cleaner targets but risking strategies the student cannot recover from observable state alone; unguided traces are noisier but grounded in the same information the student will have at deployment.

The second axis is the *amount of retained reasoning* in the training target: none, a concise rewrite, or the original long teacher reasoning. More reasoning may transfer richer planning structure, but it also lengthens sequences and may exceed what a small student model can absorb given its limited capacity. We construct all six combinations and select among them on procedural holdout alone (Section 5.1); guided prompt details are in Appendix C, and all training and evaluation prompts use the deployment-time prompt, never the hidden solution data.

3.3.2. SFT assembly. The three reasoning variants are derived from the same collected traces: the *long-reasoning* variant uses the original teacher reasoning unchanged, the *no-reasoning* variant strips reasoning while preserving the tool-call structure, and the *short-reasoning* variant uses the same teacher model to rewrite each assistant message into a strictly shorter reasoning step. For each of the resulting six trace-design combinations, we assemble balanced splits of 2,000 training and 200 validation traces, derived from 200 and 20 samples per generator. We then apply quality filters to ensure traces demonstrate genuine problem-solving: successful runs only, non-empty reasoning, rejection of *solution leakage* (hidden teacher-side reference-solution markers appearing in student-visible content), and the benchmark-exclusion scan from Section 3.2.1 applied per trace (benchmark-leakage guard). Post-hoc audits report zero solution- or benchmark-leakage matches in the assembled splits. During dataset assembly, the trace-collection system prompt is replaced with the deployment-time system prompt used for training and evaluation.

3.3.3. Stage 1 training. We fine-tune *Qwen3 4B* with low-rank adaptation (LoRA) adapters [31] using Unsloth [32] on the assembled training split. The procedural validation split is used only for checkpoint and hyperparameter selection to avoid implicit test-set overfitting to the static benchmark. Hyperparameter search follows a fixed protocol on procedural data only: a log-spaced learning-rate sweep, a LoRA-rank sweep informed by [33], and multi-seed confirmation of the selected hyperparameters.

3.4. Reinforcement Learning

A reliable security agent has to act well under its own observations, recovering from dead ends, exploring new attack paths, and reaching root within bounded interaction. SFT alone cannot deliver this: imitation of stronger teachers narrows the apparent style gap without closing the underlying capability gap [34], and training on offline teacher demonstrations leaves the policy exposed to distribution shift whenever it visits states the teacher never showed [24]. Stage 2 closes this gap by training under the policy’s own induced state distribution, using verified root as the primary signal alongside optional efficiency shaping.

3.4.1. Reward design. We seek the minimal verifier-aligned reward that reflects the operational goal of an interactive security agent without injecting task-specific privilege-escalation heuristics. We therefore introduce each reward term in isolation before combining them, following the standard reward-shaping caution that auxiliary terms can shift the optimized policy when they are not aligned with the underlying objective [35].

Outcome term. Binary verified root success is the cleanest RLVR signal, consistent with recent RLVR systems that rely on rule-based verifiers when correctness can be checked automatically [22]. For an episode e ,

$$R_{\text{out}}(e) = 2 \mathbf{1}\{\text{root}(e)\} - 1. \quad (1)$$

In a long-horizon tool-use setting, however, verified success alone under-specifies efficiency and admits wasteful successful trajectories. We therefore consider two complementary efficiency signals, each as an additive shaping term applied only on successful episodes.

Round shaping. The round term directly tracks the budgeted evaluation metric $P(H_{\text{root}} \leq r)$ and follows prior decompositions of LLM-agent efficiency into trajectory-level step efficiency and per-step cost [36]:

$$R_{\text{round}}(e) = \mathbf{1}\{\text{root}(e)\} \text{clip}_{[0,1]} \left(1 - \frac{H_{\text{root}}(e)}{h_{\text{max}}} \right), \quad (2)$$

where h_{max} is the training round budget. Round count is only a proxy for interaction cost, since one assistant round may issue multiple parallel tool calls. Optimizing the proxy can, in principle, diverge from the underlying objective, a known reward-hacking failure mode [37]. We use R_{round} at unit coefficient so the OUTCOME+ROUND variant remains interpretable as a surrogate for the round-budgeted metric.

Cost shaping. As an alternative efficiency signal, the cost term prices model inference and executed tool runtime directly:

$$R_{\text{cost}}(e) = \mathbf{1}\{\text{root}(e)\} \text{clip}_{[0,1]} \left(1 - \frac{C(e)}{C_{\text{ref}}} \right), \quad (3)$$

with episode cost

$$C(e) = \sum_{t=1}^{\tau(e)} \hat{\ell}_t^{\text{LLM}} + \sum_{t=1}^{\tau(e)} \sum_{k=1}^{m_t} \hat{\ell}_{t,k}^{\text{tool}}, \quad (4)$$

where m_t is the number of executed tool calls in round t and per-call latencies are clipped per modality, $\hat{\ell}_t^{\text{LLM}} = \min\{\ell_t^{\text{LLM}}, c_{\text{clip}}^{\text{LLM}}\}$ and $\hat{\ell}_{t,k}^{\text{tool}} = \min\{\ell_{t,k}^{\text{tool}}, c_{\text{clip}}^{\text{tool}}\}$ (see Appendix A for calibration details). Measured latency is domain-agnostic, prices parallel fan-out additively, and unlike a tool-call count cannot be gamed by collapsing several shell operations into one open-ended `exec_command`. The trade-off is that latency mixes policy behavior with infrastructure effects (server load, batching); the per-modality clip caps spikes, and rollouts within a learner group share infrastructure so relative comparisons stay roughly fair. The term is auxiliary: R_{cost} is clipped to $[0, 1]$ and weighted by λ_{cost} , fixed before RL from procedural calibration data, so cost shaping can rerank successful trajectories but cannot substitute for verified root success.

Interface guardrail. A small interface penalty is applied uniformly across all reward variants to maintain pressure on valid agent–environment interaction:

$$R_{\text{iface}}(e) = -\lambda_{\text{iface}} \mathbf{1}\{\text{iface}(e)\}. \quad (5)$$

The indicator fires at most once per episode when a non-terminal assistant response cannot be executed by the fixed harness: no executable tool call after parser normalization, malformed tool-call syntax, an unknown tool name, or schema-invalid arguments. Exact constants are reported in Appendix A. Small rule-based protocol rewards are common in RLVR and tool-use RL when the interaction contract is directly checkable [22], [38].

Reward variants. We compare four variants on procedural holdout, crossing round shaping and cost shaping in a 2×2 design over the shared R_{out} core:

$$\begin{aligned} \text{OUTCOME} &: R_{\text{out}}(e), \\ +\text{ROUND} &: R_{\text{out}}(e) + R_{\text{round}}(e), \\ +\text{COST} &: R_{\text{out}}(e) + \lambda_{\text{cost}} R_{\text{cost}}(e), \\ +\text{ROUND+COST} &: R_{\text{out}}(e) + R_{\text{round}}(e) + \lambda_{\text{cost}} R_{\text{cost}}(e). \end{aligned} \quad (6)$$

The total per-episode reward used during RL adds the guardrail uniformly to the selected variant as $R(e) = R_{\text{variant}}(e) + R_{\text{iface}}(e)$, so the iface term shifts every variant equally and does not affect their relative ranking.

3.4.2. Stage 2 training. We initialize from the selected SFT checkpoint and train with Prime-RL [39], with environment feedback exposed through a custom Verifiers [40] wrapper. The objective is group relative policy optimization (GRPO)-like [21] with four implementation differences relevant to our setting: (i) fully asynchronous rollout and learner execution with bounded staleness; (ii) no within-group reward-standard-deviation normalization; (iii) episode-level outcome rewards from direct environment verification; and (iv) a token loss with trainer-vs-inference importance ratios plus a probability-difference clip mask. Asynchrony amortizes the cost of multi-round tool-using rollouts at the price of controlled off-policyness, and procedural generators are sampled round-robin to stabilize per-generator coverage. The reward variant, the learning rate, and the final checkpoint shipped as *PrivEsc-LLM 4B* are selected on

TABLE 1. MODELS COMPARED. LOCAL OPEN-WEIGHT MODELS USE vLLM; TABLE 7 REPORTS SERVING PRECISION AND EVALUATION CONTEXT CAPS. API MODEL IDS ARE AS CURRENT AT TIME OF EVALUATION (MAY 2026).

Model	Exact ID	Deploy	Ctx
<i>Qwen3 4B</i>	Qwen3-4B-Instruct-2507 [48]	vLLM	32K
<i>Qwen3 4B SFT</i>	above + LoRA rank 8	vLLM	32K
<i>PrivEsc-LLM 4B</i>	above + LoRA rank 8	vLLM	32K
<i>Gemma 4 31B</i>	google/gemma-4-31B-it [42]	vLLM	256K
<i>DeepSeek V3.2</i>	deepseek/deepseek-v3.2 [49]	API	128K
<i>Claude Opus 4.7</i>	anthropic/claude-opus-4.7	API	200K

procedural-holdout success alone (Section 5.2); cost-term clip and reference constants are calibrated before RL from a timing benchmark of the SFT initialization, with values and procedure reported in Appendix A. The training round horizon is shorter than the evaluation budget, so transfer to the static benchmark is measured from short-horizon training to longer-horizon test-time behavior.

4. Experimental Setup

LLM models and baselines. Table 1 lists every LLM we evaluate in this work, together with their exact model identifiers, deployment method, and context length, following the recommendation of Evertz et al. [29] to reduce model-version ambiguity in LLM evaluations. *Qwen3 4B* serves as the unmodified local backbone baseline, while *Qwen3 4B SFT* and *PrivEsc-LLM 4B* are our two specialized LoRA adapters trained on top of this backbone and served locally with vLLM [41]. Beyond the Qwen3 family, we additionally compare against *Gemma 4 31B* [42] as a larger local open-weight reference point, and against *DeepSeek V3.2* [43] and *Claude Opus 4.7* as API-served frontier comparisons accessed through OpenRouter [44]. All three baseline models were selected based on Terminal-Bench [25] rankings and internal pilot evaluations. Per-model sampling parameters and context caps for each model family are reported in Table 7 of Appendix A.

Non-LLM baselines. We further compare against *ChainReactor* [45], a state-of-the-art method for automated privilege-escalation chain discovery on UNIX systems, two tool-based exploitation frameworks (Traitor [46], pwnccats [47]), and a human baseline reported in prior work [10].

Training. We fine-tune *Qwen3 4B* with quantized low-rank adaptation (QLoRA) [50] using Unsloth on 4×H100 GPUs and the filtered procedural training split described in Section 3.3. We perform the reinforcement learning step with Prime-RL on 4×H100 GPUs, initialized from the SFT checkpoint. We sample procedural generators in round-robin mode with a 20-round training horizon. We train for 1,000 steps with a batch size of 80 and 8 rollouts per instance. Table 6 in Appendix A reports the final settings.

Evaluation protocol. We evaluate each model on 10 runs per scenario across all 12 static benchmark scenarios (120 runs total), with each run capped at 60 rounds, following prior work on the same benchmark [10], and

started in a fresh container. Each model–scenario cell contains exactly 10 valid runs. The primary metric is the empirical success rate within 20 rounds, i.e., the sample estimate of $P(H_{\text{root}} \leq 20)$, and we report Wilson 95% confidence intervals throughout. All reported numbers are per-run probabilities under a fixed round budget, not best-of- k retry metrics. The static benchmark is the final external evaluation. It was not used to select the reward, the system prompt, any checkpoint, or any hyperparameter. Every such choice was made on procedural-holdout performance alone.

Cost methodology. We estimate local inference cost for the *Qwen3 4B* family from an empirical batched vLLM benchmark on an RTX 5090, and report API costs based on public OpenRouter pricing at evaluation time [44]. We normalize all per-run cost figures to expected cost per successful root at $r=20$, computed as per-run cost divided by $P(H_{\text{root}} \leq 20)$. Training-stage compute (SFT and RL) is tracked separately as one-time post-training GPU-hours on H100 hardware, priced at European on-demand H100 rates [51], and is not amortized into the per-run cost–reliability comparison. All cost figures are setup-dependent: API routing, provider price changes, batching, hardware utilization, electricity, and amortization policy can change the accounting. The full cost equations are given in Appendix A.

5. Results

Following the methodology described in Section 3 and the experimental setup described in the previous section, we address the following research questions:

- RQ1.** Which supervised fine-tuning traces best warm-start a local security agent for reinforcement learning? [Section 5.1]
- RQ2.** Which reward function design maximizes performance while avoiding reward-hacking behavior? [Section 5.2]
- RQ3.** How competitive is the resulting local agent under a fixed budgeted protocol, and where does it fail? [Section 5.3]
- RQ4.** How cost-effective is the local agent against frontier APIs, and when does post-training amortize? [Section 5.4]

We present the results in RQ order: the SFT trace-design ablation (RQ1), the RL reward-design ablation (RQ2), the budgeted static-benchmark evaluation with per-scenario and failure analysis (RQ3), and a cost–reliability comparison against API and larger local baselines (RQ4).

5.1. Trace-Design Ablation

We begin with the supervised stage that warm-starts the agent (**RQ1**): which trace design produces the strongest initialization for downstream RL?

Setup. We perform a 2×3 factorial ablation, varying two factors: trace-collection regime (guided vs. unguided) and reasoning format (none, short, or long). The ablation uses

TABLE 2. TRACE-DESIGN ABLATION ON PROCEDURAL HOLDOUT SUCCESS AT $r \leq 20$. CELLS SHOW SUCCESS RATE (SUCCESSES/500); BOLD MARKS THE SELECTED CONFIGURATION. APPENDIX TABLE 9 GIVES WILSON INTERVALS AND TRACE STATISTICS.

Trace format	Guided	Unguided
No reasoning	51.4% (257/500)	56.4% (282/500)
Short reasoning	60.2% (301/500)	58.2% (291/500)
Long reasoning	65.2% (326/500)	73.2% (366/500)

only procedural holdout performance for selection. Each condition trains the same *Qwen3 4B* SFT configuration on 2,000 balanced traces and evaluates 500 procedural holdout runs at the primary $r \leq 20$ budget.

Results. Long reasoning is the best trace format in both regimes, and unguided long-reasoning is strongest overall at 73.2%. It is at least 8.0 percentage points above every other condition and 15.0 points above the best unguided compressed-reasoning variant (Appendix Table 9 reports the Wilson 95% confidence intervals). Within the unguided regime, short reasoning adds little over no reasoning (58.2% vs. 56.4%), suggesting that retaining full observable-state reasoning matters more than adding a short reasoning field.

Interestingly, giving the teacher hidden solution data does not produce the best student: unguided traces improve procedural holdout success, consistent with training on demonstrations that match the deployment-time observable state. More open-ended exploration and recovery behavior in the unguided traces therefore appears to help the student more than solution-conditioned demonstrations. As a result, we use unguided long-reasoning traces for *Qwen3 4B SFT* in all subsequent stages.

Finding (RQ1): SFT traces should reflect the deployment-time state distribution. Unguided long-reasoning traces are strongest on procedural holdout (see Table 2). When the teacher reasons from the same observable state the student will see, the resulting traces outperform guided demonstrations with hidden solution data.

5.2. Reward-Design Ablation

With the warm-start fixed, we move to the RL stage (**RQ2**): which reward improves reliability and efficiency without inducing reward hacking?

Setup. To select the best-suited reward function, we train each of the four reward variants from Section 3.4 starting from the same SFT checkpoint, and evaluate their performance on held-out procedural runs. Each variant is trained on procedural trajectories for 1,000 steps. We evaluate checkpoints every 100 steps and use procedural success at $r \leq 20$ as the primary metric; unless stated otherwise, all reported performance figures refer to this metric. Only if multiple reward variants perform on par, we use $r \leq 60$ as a tie-breaker. Note that we perform reward selection entirely on procedural holdouts, and never use static benchmark results to select the reward, prompt, or hyperparameters.

Results. Figure 3 depicts the results. The vanilla OUTCOME-only reward function shows already strong per-

	No cost term	Cost term
No round term	OUTCOME step 200 $r \leq 5$ 50% $r \leq 20$ 88% $r \leq 60$ 90% S_{all} 6/10	+COST + step 300 $r \leq 5$ 67% $r \leq 20$ 90% $r \leq 60$ 92% S_{all} 7/10
	+ROUND step 400 $r \leq 5$ 80% $r \leq 20$ 90% $r \leq 60$ 90% S_{all} 5/10	+ROUND+COST step 200 $r \leq 5$ 63% $r \leq 20$ 87% $r \leq 60$ 87% S_{all} 5/10
Round term		

Figure 3. Reward-factor ablation on procedural holdout. Cells show the checkpoint selected by the procedural rule: primary $P(H_{root} \leq 20)$, with $P(H_{root} \leq 60)$ recovery and step-1000 stability used only as tie-breakers. Values report success at $r \leq 5, 20, 60$ and S_{all} , the count of procedural scenario families solved on every rollout within the max budget.

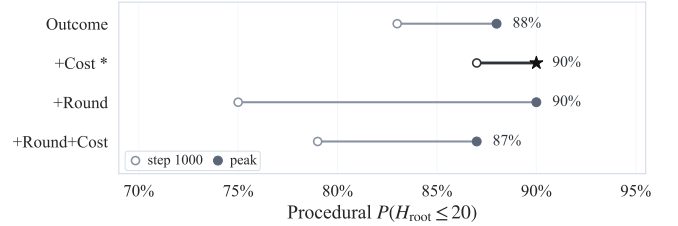


Figure 4. Reward-stability comparison on procedural holdout at the primary budget. Lines connect step-1000 performance to the checkpoint selected under the procedural rule; the x-axis shows $P(H_{root} \leq 20)$.

formance: root verification alone reaches 88.0% (90.0% at $r \leq 60$), improving sharply over the selected SFT initialization with only 73.2% (see Appendix Table 9). OUTCOME+COST and OUTCOME+ROUND show the best performance with 90.0% success rate. However, OUTCOME+COST shows the highest success rate at $r \leq 60$.

Additionally, OUTCOME+COST also maintains the strongest primary-budget stability for the OUTCOME signal as depicted in Figure 4. In contrast to the other variants, it keeps the signal strong while pricing LLM and tool latency directly, retaining most of the outcome gain without relying on a round-count proxy. For instance, while OUTCOME+ROUND improves early success, reaching 90.0% at both $r \leq 20$ and $r \leq 60$, its performance degrades afterwards, ending below OUTCOME at the primary $r \leq 20$ budget. We interpret this as proxy-optimization behavior consistent with reward hacking [37]: explicit round pressure improves apparent efficiency early, but the shape is consistent with shortcut learning [52]. The agent favors short episodes and loses the longer recovery loops needed for robust transfer. These instability and overfitting trends appear on procedural holdout, whereas training performance remains high and stable for all variants. Based on these results, we select OUTCOME+COST as our final reward function.

Finding (RQ2). Price cost, not rounds. Round bonuses reward short episodes but ignore tool calls inside each round. OUTCOME+ROUND peaks early, then drifts below the unshaped OUTCOME baseline; only OUTCOME+COST, which prices LLM and tool latency additively, remains stable on the holdout across training.

5.3. Budgeted Static-Benchmark Results

With trace and reward design fixed on procedural holdout, we turn to the static benchmark, the final external evaluation untouched by selection (RQ3): how does the 4B policy compare to frontier API and larger open-weight baselines under tight budgets, and where does it fail?

Setup. Each model runs 10 times on each of the 12 static benchmark scenarios (120 runs total), in fresh containers capped at 60 rounds. The primary metric is $P(H_{\text{root}} \leq 20)$.

LLM results. Figure 1 reports per-stage success of different models on the static benchmark. *PrivEsc-LLM 4B* solves 112 runs of 120 runs, reaching a performance of 93.3%. In contrast, the vanilla *Qwen3 4B* reaches only 40.8% (49/120). SFT on the selected unguided long-reasoning traces lifts *Qwen3 4B SFT* to 79.2% (95/120), and RL with OUTCOME+COST further improves it to 93.3%, yielding *PrivEsc-LLM 4B*. While *Claude Opus 4.7* solves all 120 runs and sets the empirical ceiling, *PrivEsc-LLM 4B* outperforms it under a tight budget of $r=5$, reaching 78.3% versus Claude’s 66.7%. The two post-training stages contribute asymmetrically: SFT provides the large capability jump from the backbone, while RL closes much, but not all, of the remaining gap to Claude.

Non-LLM results. We also compare against related methods and human baselines. First, we evaluate *ChainReactor* [45] as a symbolic-planning baseline. Under a 30-minute per-scenario budget, it returns a valid plan for 1 of 12 scenarios (*SUID GTFOBins*); the rest end in no plan or planner timeout. We attribute this to a representation-coverage gap rather than a planning failure: most benchmark scenarios hinge on primitives (e.g., credential discovery, sudoers semantics, or container/host boundaries) that the extractor and PDDL domain do not express, so no wall-clock budget closes the gap. This underscores a central lesson for agentic systems: planners are only as strong as the state abstractions their harness exposes.

We further compare against the human and traditional-tool numbers reported by Happe et al. [10] on the benchmark dataset. *PrivEsc-LLM 4B* exceeds the reported human baseline ($\sim 75\%$) and substantially surpasses the traditional-tools baseline ($\sim 25\%$). Note that these numbers reflect benchmark-context comparisons only and were not rerun under our repeated-run budgeted protocol.

Per-scenario analysis. Figure 5 breaks down performance on a per-scenario basis. All counts use the same primary budget $r \leq 20$, where most models plateau.

Among the locally trained models, *Qwen3 4B SFT* is already strong on credential and file scenarios but drops on scenarios that need precise multi-step exploitation: *Weak root password* at 5/10, *Cron wildcard injection* at 4/10, and *Docker group escape* at 0/10. RL closes these gaps



Figure 5. Per-scenario success at the primary $r \leq 20$ budget, where each cell shows $x/10$ successful runs and color encodes the success rate.

everywhere except Docker, with the largest gains over SFT on the cells that demand reliable multi-step exploit construction. *PrivEsc-LLM 4B* scores 10/10 on all 10 scenarios covered by procedural families, and even 10/10 on *Sudo no password* despite the absence of a direct procedural analog. The sole remaining gap is *Docker group escape*, the withheld-family scenario: its low but non-zero success rate reflects brittle out-of-distribution generalization, as it requires reasoning about container runtime internals and indirect privilege relationships that no training procedure explicitly covered.

Among the external models, *Claude Opus 4.7* scores 10/10 across all 12 scenarios, including *Docker group escape*, the one scenario withheld from both procedural training profiles. By contrast, *DeepSeek V3.2* achieves 10/10 on only 1 of 12 scenarios and fails entirely on three scenarios, showing that frontier scale alone does not guarantee reliable exploit construction or generalization.

Failure analysis. *PrivEsc-LLM 4B* fails 8 of 120 headline runs, all on *Docker group escape*. The Docker headline cell contains 2 successes at $r \leq 20$ and 3 at $r \leq 60$ out of 10 runs (annotated trace excerpts appear in Appendix B).

Inspecting the failed traces shows a more specific limit than failure to recognize the vulnerability. The policy identifies Docker and issues Docker commands in all failed traces, and reaches root inside a child container in most of them. The common failure is treating container-root evidence as progress and not turning it into a verifier-accepted root shell on the target. One failed trace tries the target-root style `-v /:/host mount` and none executes `chroot`; the rest stay in container-root or credential-check loops. The larger *Gemma 4 31B* baseline solves the same

Docker scenario 10/10, suggesting a capability gap in the 4B base model: without Docker-family training, RL finds little reliable Docker behavior to amplify.

Finding (RQ3). Recipe transfers; Docker remains the gap. At $r=20$, *PrivEsc-LLM 4B* reaches 93.3% vs. 100% for *Claude Opus 4.7* at $\sim 82\times$ lower cost, leading at $r=5$ (78.3% vs. 66.7%). All headline failures are *Docker group escape*, the only benchmark scenario withheld as a full procedural family.

5.4. Training and Inference Cost

With reliability settled, we finally turn to deployment economics (RQ4): how does the local agent’s cost-reliability operating point compare to API and larger local baselines, and when does post-training amortize?

Setup. We report expected cost per successful root at $r=20$, computed as per-run cost divided by $P(H_{\text{root}} \leq 20)$. Local inference cost is anchored to a batched vLLM benchmark on an RTX 5090; API cost uses public OpenRouter pricing at evaluation time [44]. Post-training compute is tracked separately as a one-time H100 GPU spend at European on-demand rates [51] and is not amortized into per-run figures. Full equations are in Appendix A.

Results. Post-training moves the 4B ladder toward the upper-left region of Figure 6: *Qwen3 4B* reaches 40.8% at \$0.00414 per successful root, *Qwen3 4B SFT* 79.2% at \$0.00269, and *PrivEsc-LLM 4B* 93.3% at \$0.00213. *PrivEsc-LLM 4B* trails *Claude Opus 4.7* by 6.7 pp in reliability but is about $82.6\times$ cheaper per successful root (\$0.00213 vs. \$0.1759), and $5.5\times$ cheaper than *Gemma 4 31B* while exceeding its 80.8% success rate by 12.5 pp. *DeepSeek V3.2* reaches 52.5% at \$0.0285 per successful root, so its long-budget recovery does not translate into an efficient primary-budget operating point. Post-training itself is a one-time \$120.80 GPU spend, amortizing against *Claude Opus 4.7* after roughly 700 successful escalations at $r=20$.

Finding (RQ4). Post-training amortizes after ~ 700 escalations. *PrivEsc-LLM 4B* sits in the upper-left of the cost-reliability frontier: 93.3% at \$0.00213 per successful root, about $82\times$ cheaper than *Claude Opus 4.7* at the same budget. The one-time \$120.80 post-training spend amortizes against *Claude Opus 4.7* after roughly 700 successful escalations at $r=20$.

6. Discussion

Our results demonstrate that the proposed recipe can train strong local models for Linux privilege escalation. In the following, we discuss the main lessons from our evaluation, additional insights from the ablations and audits, and the broader applicability of the approach.

Ablation Insights. The two ablations point at specific choices, not at SFT or RL as a whole. For SFT, the trace-design ablation finds that traces collected without hidden solution data, but with long observable-state reasoning, close most of the gap to the deployment-time distribution; compressed reasoning gives back only a fraction. For RL, round shaping, alone or with cost shaping, peaks early and then

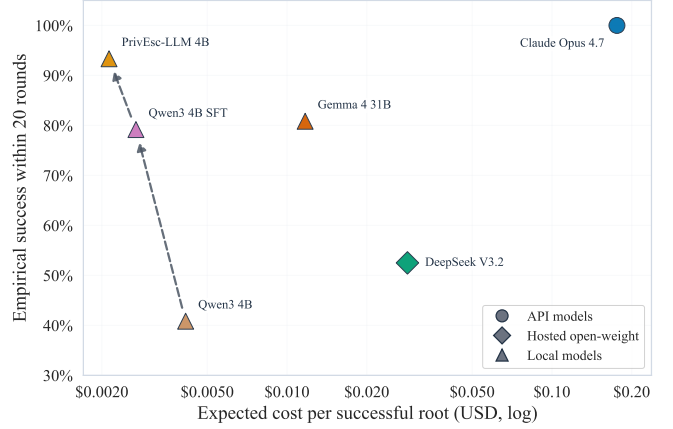


Figure 6. Static-benchmark reliability at $r \leq 20$ rounds versus expected cost per successful root at the same budget, over 120 runs per model. Cost is expected per-run cost divided by $P(H_{\text{root}} \leq 20)$, so the desirable region is upper left. Local points use effective input/output token prices anchored to measured batched serving cost; API points use provider token prices observed at evaluation time. The Qwen post-training progression moves from the backbone to SFT and then RL.

drifts back below the unshaped OUTCOME baseline; only OUTCOME+Cost holds its peak through 1000 steps.

Cost Amortization. Post-training requires a meaningful upfront investment, while API-based frontier models currently offer relatively convenient and cost-effective access. However, post-training a local model remains attractive not only for data protection and deployment control in enterprise settings, but also because the investment can amortize surprisingly quickly at moderate usage volumes. Treated as a fixed setup cost, post-training becomes economically appealing under our local-accounting assumptions (European on-demand H100 pricing [51]): the one-time of \$120.80 amortizes against *Claude Opus 4.7* after roughly 700 successful escalations at $r=20$.

Prompt Sensitivity. A post-hoc audit reruns the static benchmark under three predeclared system-prompt variants without changing model selection, headline prompts, or figures. The raw *Qwen3 4B* is highly prompt-sensitive: at $r \leq 20$, success drops from 42.1% with the detailed prompt to 25.4% with the budget prompt and 21.7% with the minimal prompt. The trained Qwen systems are much less sensitive in the same audit: *Qwen3 4B SFT* spans 75.8–80.8% and *PrivEsc-LLM 4B* spans 88.3–91.7%. *Gemma 4 31B* is also stable (77.1–82.5%), with the budget prompt slightly outperforming the detailed prompt. This audit does not prove prompt invariance, but it checks that the Qwen post-training gains are not an artifact of one verbose prompt. We report more details in Table 11.

Beyond Privilege Escalation. Although we focus on Linux privilege escalation, any task with a binary, environment-verifiable outcome and a tool-use loop is a plausible candidate for adoption, such as crash reproduction or patch validation [2], [1]. Linux privilege escalation is a particularly clean starting point because success is unambiguous, difficulty scales across exploit families, and

the environment resets reliably, making it an ideal setting to isolate the effect of each training design choice before moving to noisier domains. Exploring the recipe across such domains is a natural next step.

7. Threats to Validity

Our evidence is limited to one setting and one base architecture, *Qwen3 4B*, so cross-family generality remains open. The API comparisons in Table 1 are limited to the specific configurations evaluated, so we cannot draw certain conclusions about broader model families [29]. The RL training phase required 4×H100 GPUs for about 11.5 hours to reach the deployed checkpoint, so the commodity-hardware claim applies to inference, not to post-training process itself.

The procedural generators cover common misconfiguration families but not the long tail of real-world escalation paths; *Docker group escape* is the concrete instance. Because RL post-training largely amplifies behaviors already present in the base distribution [53], a multi-step exploit chain that is underrepresented in the 4B backbone’s pretraining and absent from procedural training is hard to recover. The *Gemma 4 31B* 10/10 result on the same scenario does not show that our recipe would scale to a larger backbone; it only places the Docker gap at the 4B base model’s capabilities under the evaluated setup.

Verifiable-reward gains are not robust to natural shaping choices and must be re-validated whenever the reward is changed [37]. Evaluation artifacts can still occur (e.g., terminal output truncation), so trace-level anomaly checks remain necessary; exhaustive per-model prompt optimization and API prompt sensitivity remain future work.

8. Related Work

LLM agents for penetration testing. Prior LLM pentest agents show that LLMs can act in offensive-security loops, but they rarely combine local deployment, leakage-controlled generalization, and strict repeated-run budgeted evaluation. Early work showed that LLMs can plan and execute autonomous Linux privilege escalation [54], with follow-ups scaling the offensive-loop framing to enterprise Active Directory networks [55]. PentestGPT [56] introduced a three-module agent and a real-world pentest benchmark. Closest to our setting is the controlled Linux privilege-escalation benchmark we build on [10], previously studied across multiple cloud-hosted LLMs. Compared with this and other prior LLM-pentest agents, our setting (i) replaces cloud-hosted frontier models with a post-trained small local model, (ii) isolates leakage-controlled generalization via value-disjoint procedural training, and (iii) evaluates under a strict budgeted, repeated-run protocol.

Cybersecurity benchmarks. Most cybersecurity benchmarks emphasize breadth; fewer center value-disjoint generalization, fixed interaction budgets, repeated runs, and confidence intervals. CyBench [9] evaluates agents on 40

capture the flag (CTF) tasks with subtask scoring, AutoPenBench [57] introduces milestone-based scoring on 33 vulnerability-testing tasks, and CyberGym [2] scales execution-based reproduction to 1,507 real-world vulnerabilities across 188 projects. We focus on host-level Linux privilege escalation specifically because its success is automatically verifiable, which is what enables clean reward signals and a budgeted primary metric.

Post-training with verifiable rewards. RLVR has improved multi-step reasoning on math and coding tasks when correctness can be verified automatically [21], [23], [22]. ToolRL [38] studies structured reward design for tool-use RL and shows that coarse final-answer rewards are often insufficient when actions include tool selection and parameterization. CTF-Dojo [58] trains CTF agents on execution-verified trajectories but relies on SFT rather than RL. Privilege escalation is a clean RLVR setting because whether the agent reached root, how many rounds it used, and whether its tool calls were valid are all visible in the trace. To the best of our knowledge, no prior work applies RLVR to an interactive security task; we fill this gap with a host-exploitation loop whose success is automatically verifiable from tool outcomes.

9. Conclusion

We presented a leakage-controlled post-training recipe for turning a small open-weight language model into a reliable tool-using security agent for Linux privilege escalation. Starting from *Qwen3 4B*, supervised fine-tuning on procedurally generated traces raises success on the held-out benchmark from 40.8% to 79.2% within 20 rounds, while reinforcement learning with verifiable rewards further improves our model to reach 93.3%. These gains show that small local models can approach frontier-model reliability on interactive security tasks when training preserves the deployment-time tool interface and uses automatically verifiable outcomes.

The resulting 4B agent is competitive and economical: it exceeds the reported human baseline at the primary budget, leads the frontier ceiling at $r=5$ (78.3% vs. 66.7%), and costs about 82× less per successful root than *Claude Opus 4.7*. Under our local-accounting assumptions, post-training amortizes after roughly 700 successful escalations. The recipe and protocol behind these numbers are the contribution; they are natural candidates for other verifier-friendly security tasks.

10. Ethical Considerations

This work trains a model to escalate privileges, so the dual-use risk is real. We mitigate that risk by restricting scenarios to already documented Linux misconfigurations and known privilege-escalation paths, running all experiments in isolated containers, and evaluating within a bounded benchmark rather than open-ended offensive tasks. The experiments do not interact with live third-party systems, collect

personal data, or involve human subjects; accordingly, the work was outside the scope of institutional human-subjects review. Responsible disclosure to vendors is not applicable because the paper does not introduce new vulnerabilities or exploit primitives; its contribution is a post-training recipe for acting more reliably on public ones.

Acknowledgments

This work was supported by the Vienna Science and Technology Fund (WWTF) under project BREADS (10.47379/VRG23011). Experiments ran on the MUSICA cluster provided by Austrian Scientific Computing (ASC).

References

- [1] C. Zhang, Y. Park, F. Fleischer, Y.-F. Fu, J. Kim, D. Kim, Y. Kim, Q. Xu, A. Chin, Z. Sheng *et al.*, “SoK: DARPA’s AI cyber challenge (AIXCC): Competition design, architectures, and lessons learned,” 2026, arXiv:2602.07666.
- [2] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song, “CyberGym: Evaluating AI agents’ real-world cybersecurity capabilities at scale,” in *Proc. of the Int. Conference on Learning Representations (ICLR)*, 2026.
- [3] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, “Fuzz4All: Universal fuzzing with large language models,” in *Proc. of the IEEE/ACM Int. Conference on Software Engineering (ICSE)*, 2024.
- [4] OpenSSL Project, “Security vulnerabilities and advisories,” Vulnerability page, 2026, accessed 2026-03-05. [Online]. Available: <https://www.openssl.org/news/vulnerabilities.html>
- [5] B. Grinstead, C. Holler, and F. Braun, “Behind the scenes hardening Firefox with Claude Mythos Preview,” Mozilla Hacks blog post, May 2026, accessed 2026-05-25. [Online]. Available: <https://hacks.mozilla.org/2026/05/behind-the-scenes-hardening-firefox/>
- [6] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, “Extracting training data from large language models,” in *Proc. of the USENIX Security Symposium*, 2021.
- [7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. of the Int. Conference on Learning Representations (ICLR)*, 2023.
- [8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] A. K. Zhang, N. Perry, R. Dulepet, J. Ji, C. Menders, J. W. Lin, E. Jones, G. Hussein, S. Liu, D. J. Jasper, P. Peetathawatchai, A. Glenn, V. Sivashankar, D. Zamoshchin, L. Glikbarg, D. Askaryar, H. Yang, A. Zhang, R. Alluri, N. Tran *et al.*, “CyBench: A framework for evaluating cybersecurity capabilities and risks of language models,” in *Proc. of the Int. Conference on Learning Representations (ICLR)*, 2025.
- [10] A. Happe, A. Kaplan, and J. Cito, “LLMs as hackers: Autonomous Linux privilege escalation attacks,” *Empirical Software Engineering*, vol. 31, no. 3, 2026.
- [11] GTF0Bins, “GTF0Bins,” GitHub repository, 2026, accessed 2026-03-11. [Online]. Available: <https://github.com/GTF0Bins/gtfobins>. github.io
- [12] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [13] F. Petroni, T. Rocktäschel, P. Lewis, A. Bakhtin, Y. Wu, A. H. Miller, and S. Riedel, “Language models as knowledge bases?” in *Proc. of the Conference on Empirical Methods in Natural Language Processing and the Int. Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- [14] A. Roberts, C. Raffel, and N. Shazeer, “How much knowledge can you pack into the parameters of a language model?” in *Proc. of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [15] H. Chang, J. Park, S. Ye, S. Yang, Y. Seo, D.-S. Chang, and M. Seo, “How do large language models acquire factual knowledge during pretraining?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [16] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V. Zhao, Y. Huang, A. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V. Le, and J. Wei, “Scaling instruction-finetuned language models,” *Journal of Machine Learning Research (JMLR)*, vol. 25, 2024.
- [17] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [18] A. Yang, A. Li, B. Yang *et al.*, “Qwen3 technical report,” 2025, arXiv:2505.09388.
- [19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2018.
- [20] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” in *Advances in Neural Information Processing Systems (NIPS)*, 2017.
- [21] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” 2024, arXiv:2402.03300.
- [22] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning,” *Nature*, vol. 645, 2025.
- [23] X. Wen, Z. Liu, S. Zheng, S. Ye, Z. Wu, Y. Wang, Z. Xu, X. Liang, J. Li, Z. Miao, J. Bian, and M. Yang, “Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base LLMs,” in *Proc. of the Int. Conference on Learning Representations (ICLR)*, 2026.
- [24] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. of the Int. Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [25] M. A. Merrill, A. G. Shaw, N. Carlini *et al.*, “Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces,” 2026, arXiv:2601.11868.
- [26] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, “Leveraging procedural generation to benchmark reinforcement learning,” in *Proc. of the Int. Conference on Machine Learning (ICML)*, 2020.

- [27] C. Packer, K. Gao, J. Kos, P. Krähenbühl, V. Koltun, and D. Song, “Assessing generalization in deep reinforcement learning,” in *Proc. of the Int. Conference on Learning Representations (ICLR)*, 2019.
- [28] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *Proc. of the USENIX Security Symposium*, 2022.
- [29] J. Evertz, N. Risse, N. Neuer, A. Müller, P. Normann, G. Sapia, S. Gupta, D. Pape, S. Shaw, D. Srivastav, C. Wressnegger, E. Quiring, T. Eisenhofer, D. Arp, and L. Schönherr, “Chasing shadows: Pitfalls in LLM security research,” in *Proc. of the Network and Distributed System Security Symposium (NDSS)*, 2026.
- [30] C. Deng, Y. Zhao, X. Tang, M. Gerstein, and A. Cohan, “Investigating data contamination in modern benchmarks for large language models,” in *Proc. of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2024.
- [31] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. of the Int. Conference on Learning Representations (ICLR)*, 2022.
- [32] D. Han, M. Han, and Unsloth team, “Unsloth,” GitHub repository, 2023, accessed 2026-03-05. [Online]. Available: <https://github.com/unslothai/unsloth>
- [33] J. Schulman and Thinking Machines Lab, “LoRA without regret,” *Thinking Machines Lab: Connectionism*, 2025.
- [34] A. Gudibande, E. Wallace, C. V. Snell, X. Geng, H. Liu, P. Abbeel, S. Levine, and D. Song, “The false promise of imitating proprietary language models,” in *Proc. of the Int. Conference on Learning Representations (ICLR)*, 2024.
- [35] A. Y. Ng, D. Harada, and S. J. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. of the Int. Conference on Machine Learning (ICML)*, 1999.
- [36] S. Chen, M. Zhao, L. Xu, Y. Zhao, B. Zhu, H. Zhang, S. Zhao, and C. Lu, “DEPO: Dual-efficiency preference optimization for LLM agents,” in *Proc. of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026.
- [37] J. Skalse, N. H. R. Howe, D. Krashennnikov, and D. Krueger, “Defining and characterizing reward gaming,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [38] C. Qian, E. C. Acikgoz, Q. He, H. Wang, X. Chen, D. Hakkani-Tür, G. Tur, and H. Ji, “ToolRL: Reward is all tool learning needs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [39] Prime Intellect, “PRIME-RL,” GitHub repository, 2025, accessed 2026-03-04. [Online]. Available: <https://github.com/PrimeIntellect-ai/prime-rl>
- [40] W. Brown, “Verifiers: Environments for LLM reinforcement learning,” GitHub repository, 2025, accessed 2026-03-05. [Online]. Available: <https://github.com/PrimeIntellect-ai/verifiers>
- [41] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [42] Google DeepMind, “Gemma 4 31B IT model card,” Hugging Face model card, 2026, accessed 2026-05-22. [Online]. Available: <https://huggingface.co/google/gemma-4-31b-it>
- [43] DeepSeek-AI, A. Liu, A. Mei *et al.*, “DeepSeek-V3.2: Pushing the frontier of open large language models,” 2025, arXiv:2512.02556.
- [44] OpenRouter, “Pricing,” Pricing page, 2026, accessed 2026-03-08. [Online]. Available: <https://openrouter.ai/pricing>
- [45] G. D. Pasquale, I. Grishchenko, R. Iesari, G. Pizarro, L. Cavallaro, C. Kruegel, and G. Vigna, “ChainReactor: Automated privilege escalation chain discovery via AI planning,” in *Proc. of the USENIX Security Symposium*, 2024.
- [46] L. Galvin, “Traitor: Automatic Linux privilege escalation,” GitHub repository, 2021, accessed 2026-03-10. [Online]. Available: <https://github.com/liamg/traitor>
- [47] C. Stewart, “pwncat,” GitHub repository, 2021, accessed 2026-03-10. [Online]. Available: <https://github.com/calebstewart/pwncat>
- [48] Qwen Team, “Qwen3-4B-Instruct-2507 model card,” Hugging Face model card, 2025, accessed 2026-05-25. [Online]. Available: <https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>
- [49] DeepSeek-AI, “DeepSeek-V3.2 model card,” Hugging Face model card, 2025, accessed 2026-05-25. [Online]. Available: <https://huggingface.co/deepseek-ai/DeepSeek-V3.2>
- [50] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [51] Verda, “Premium GPU instances,” Pricing page, 2026, accessed 2026-03-07. [Online]. Available: <https://verda.com/products>
- [52] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, no. 11, 2020.
- [53] R. Zhao, A. Meterez, S. M. Kakade, C. Pehlevan, S. Jelassi, and E. Malach, “Echo chamber: RL post-training amplifies behaviors learned in pretraining,” in *Proc. of the Conference on Language Modeling (COLM)*, 2025.
- [54] A. Happe and J. Cito, “Getting pwn’d by AI: Penetration testing with large language models,” in *Proc. of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023.
- [55] —, “Can LLMs hack enterprise networks? autonomous assumed breach penetration-testing active directory networks,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 6, 2026.
- [56] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, “PentestGPT: Evaluating and harnessing large language models for automated penetration testing,” in *Proc. of the USENIX Security Symposium*, 2024.
- [57] L. Gioacchini, A. Delsanto, I. Drago, M. Mellia, G. Siracusano, and R. Bifulco, “AutoPenBench: A vulnerability testing benchmark for generative agents,” in *Proc. of the Conference on Empirical Methods in Natural Language Processing: Industry Track (EMNLP Industry Track)*, 2025.
- [58] T. Y. Zhuo, D. Wang, H. Ding, V. Kumar, and Z. Wang, “Training language model agents to find vulnerabilities with CTF-Dojo,” 2025, arXiv:2508.18370.
- [59] Eurostat, “Electricity prices for non-household consumers, second half 2025,” Dataset `nrg_pc_205`, EU average including non-recoverable taxes and levies, 2026, accessed 2026-05-24. [Online]. Available: https://ec.europa.eu/eurostat/databrowser/view/nrg_pc_205/default/table

Appendix

A Additional Reproducibility Details

Reward Function Details

We compare four verifier-aligned reward variants while keeping the training pipeline fixed, crossing round shaping and cost shaping in a 2×2 design: `outcome`, `outcome_round`, `outcome_cost`, and `outcome_round_cost`. `outcome_cost` is selected for *PrivEsc-LLM 4B* based on procedural ablation results (Section 5.2). The cost term $C(e)$ is defined by Equation 4. In the final training configuration we use $h_{\max}=20$,

TABLE 3. PROCEDURAL GENERATOR FAMILIES USED IN TRAINING AND VALIDATION. THE TABLE SUMMARIZES THE 10 SEEDED PROCEDURAL ENVIRONMENTS USED FOR TRACE COLLECTION, SFT DATA ASSEMBLY, AND RL TRAINING. PER-SCENARIO BENCHMARK EXCLUSIONS ARE LISTED IN TABLE 4.

Generator	Description
SUID GTFOBins	Places a non-interactive GTFOBins binary on disk with the SUID bit set.
Sudo GTFOBins	Grants passwordless sudo access to a non-interactive GTFOBins binary.
File capabilities	Assigns Linux file capabilities to a GTFOBins binary.
Password-history leakage	Leaks reusable credentials through shell history.
Password-file leakage	Stores reusable credentials in files.
Password reuse	Reuses a low-privilege credential as the root password.
Weak root password	Samples the root password from a weak-password distribution.
Cron wildcard injection	Creates a cron-driven archive workflow vulnerable to wildcard injection.
Writable cron script	Exposes a writable cron-executed script.
SSH key reuse	Reuses an existing private key for root authentication.

$\lambda_{\text{cost}}=0.1$, $\lambda_{\text{iface}}=0.05$, $c_{\text{clip}}^{\text{llm}}=20000$ ms, $c_{\text{clip}}^{\text{tool}}=65000$ ms, and $C_{\text{ref}}=540000$ ms. All values were fixed before RL from H100 calibration runs under training-like inference concurrency; see Reward-Cost Calibration below for the calibration source and evidence. The interface penalty fires at most once per episode when a non-terminal assistant response cannot be executed by the fixed harness: no executable tool call after parser normalization, malformed tool-call syntax, an unknown tool name, or schema-invalid arguments.

Reward-Cost Calibration

All cost constants are fixed before RL from a timing benchmark of the selected SFT adapter under training-like H100 concurrency (100 traces, 74 successful). Success-conditioned per-call latencies have LLM $P_{99}=18.1$ s and cron-tool $P_{99}=30.9$ s, motivating $c_{\text{clip}}^{\text{llm}}=20$ s and $c_{\text{clip}}^{\text{tool}}=65$ s; these clip 0.60% of successful LLM calls and 0.11% of tool calls. Successful clipped episode cost has $P_{99}=272$ s, so $C_{\text{ref}}=540$ s ($\approx 2 \times P_{99}$, rounded to 10 s) leaves RL fan-out headroom while preserving cost pressure; 0/74 calibration successes saturate, and post-hoc training rollouts show a 0.38% saturation rate.

Teacher Inference Configuration

The supervised trace-collection teacher and the short-reasoning rewriter share a single model and provider route; Table 8 reports their request-level settings. All requests use the OpenAI-compatible Chat Completions API and pin the OpenRouter provider to first-party DeepSeek (`provider.only=[deepseek]`, `provider.allow_fallbacks=false`). Sampling parameters not listed are not set on the request, so trace-collection rollouts sample under DeepSeek-routed OpenRouter defaults.

Cost Methodology

Local inference cost is estimated as serving-only batched vLLM cost on an RTX 5090 under corporate local-deployment assumptions, including hardware amortization and electricity but excluding agent orchestration, Docker/-

tool latency, model download, and server cold start. We amortize a \$1,999 RTX 5090 plus \$700 host system over 3 years at 50% utilization (\$0.205/hr), and price 0.675 kW total draw at the Eurostat H2 2025 EU non-household electricity rate of €0.1837/kWh [59], converted at the ECB EUR/USD reference rate to \$0.213/kWh (\$0.144/hr). The resulting hardware-second cost is $C_{\text{sec}}=9.70 \times 10^{-5}$ \$/s.

Because vLLM co-schedules prefill and decode under batching, we report a mixed-workload token price calibrated at the throughput/latency knee (concurrency 16):

$$C_{\text{mix}}^{\text{1M}} = C_{\text{sec}} \frac{T_{\text{bench}}}{N_{\text{in,bench}} + N_{\text{out,bench}}} \cdot 10^6. \quad (7)$$

For Qwen3-4B serving this gives $C_{\text{mix}}^{\text{1M}}=\$0.0323/1\text{M}$ tokens. Per-model input/output prices used by the pricing utility are fit by replaying real per-round prompts at the same concurrency and anchored to this mixed cost, preserving the measured aggregate serving price. Expected cost per successful root at budget R is

$$C_{\text{root}}(R) = \frac{\mathbb{E}[C_{\text{run}}(R)]}{P(H_{\text{root}} \leq R)}. \quad (8)$$

Under this accounting, *PrivEsc-LLM 4B* costs \$0.00213 per successful root at $r \leq 20$. API costs use publicly listed input and output token prices at evaluation time via OpenRouter [44]; *Claude Opus 4.7* costs \$0.1759 per successful root at the same budget. Training-stage cost uses the Verda H100 SXM5 on-demand rate of \$2.29 per GPU-hour [51] applied to the measured wall-clock GPU-hours in Table 6. These prices can change by provider, route, and date, while local costs depend on hardware utilization, batching, electricity, and amortization policy. All reported cost figures are therefore setup-specific.

B Interaction-Policy Examples

To ground the trace-level discussion in the main text, we release two *PrivEsc-LLM 4B* static-benchmark transcripts in the artifact repository¹ under `examples/`; each contains the full system prompt, every assistant message, and every tool call/result, with a short header (model, scenario, success flag, turn count). The success run is `rl_success_cron_wildcard_trace.txt` and the failure run is `rl_failure_docker_trace.txt`.

Success, Cron wildcard injection (4 rounds). R1 issues 9 parallel recon calls (`id, sudo -l, SUID` and `getcap` scans, `crontab` listing, `ps, sudoers`) and discovers a root-owned `/etc/cron.d/backup_lowpriv`. R2 reads the cron job and writable directories, identifying a `tar -zcf ...` * pattern over `/home/lowpriv/backup`. R3 plants the wildcard payload (`shell.sh, --checkpoint=1, --checkpoint-action=exec=sh shell.sh`) so the next cron tick `SUID-flags /bin/bash`. R4 opens an interactive root shell via `bash -p`.

Failure, Docker group escape (60 rounds, withheld family). R1 finds docker-group membership; from R2 onward the policy issues over 1,000 docker run

TABLE 4. BENCHMARK SCENARIO TAXONOMY: THE 12 STATIC BENCHMARK SCENARIOS, MAPPED TO THEIR CLOSEST PROCEDURAL GENERATOR FAMILY (TABLE 3) AND LABELED *covered* (VULNERABILITY CLASS IN TRAINING, BENCHMARK-SPECIFIC VALUES EXCLUDED), *no direct procedural analog*, OR *withheld family*. ALL SCENARIOS ALSO INHERIT THE UNIVERSAL `LOWPRIV` AND `/HOME/LOWPRIV/` EXCLUSIONS.

Static benchmark scenario	Closest procedural family	Distribution status	Benchmark-specific values excluded from sampling or audited absent
SUID GTFOBins	SUID GTFOBins	covered	Binaries <code>find</code> , <code>python</code> , <code>python3</code> , <code>python3.11</code>
Password-history leakage	Password-history leakage	covered	Password <code>aim8Du7h</code> ; pair <code>root:aim8Du7h</code> ; three exact history-leakage snippets (<code>sshpas</code> , <code>echo su root, expect</code>)
Sudo no password	—	no direct procedural analog	No generator samples unrestricted <code>NOPASSWD:ALL</code> ; the restricted <i>Sudo GTFOBins</i> family is separate. Sudoers filename <code>allow-all</code> and exact <code>NOPASSWD:ALL</code> string audited absent from assembled traces.
Sudo GTFOBins	Sudo GTFOBins	covered	Binary <code>tar</code> ; sudoers filename <code>/etc/sudoers.d/gtfo-tar</code> ; <code>NOPASSWD:/usr/bin/tar</code> string
Docker group escape	—	withheld family	Entire Docker group-escape family withheld; no procedural instance exposes docker-group membership or a Docker socket, and host-bind-mount solve fragments are audited absent from training traces.
Password reuse (MySQL)	Password-file leakage	covered	Password <code>aim8Du7h</code> ; pair <code>root:aim8Du7h</code> ; filename <code>.my.cnf</code>
Password reuse	Password reuse	covered	Password <code>trustnol</code> ; pair <code>root:trustnol</code>
Weak root password	Weak root password	covered	Password <code>root</code> removed from the common-password pool
Root allows lowpriv SSH	SSH key reuse	covered	SSH key name <code>id_rsa</code>
Cron wildcard injection	Cron wildcard injection	covered	Cron job name <code>backup_lowpriv</code> ; backup removed from <code>cron_backup_dirs</code> ; <code>/home/*/backup</code> paths
Writable cron script	Writable cron script	covered	Cron job name <code>lowpriv_script</code> ; <code>cron.sh</code> removed from <code>cron_script_names</code>
Password-file leakage	Password-file leakage	covered	Password <code>aim8Du7h</code> ; pair <code>root:aim8Du7h</code> ; filename <code>admin_on_vacation.txt</code>

TABLE 5. TRAINING-STAGE SELECTION PROTOCOL USED BEFORE STATIC-BENCHMARK EVALUATION.

Stage	Selection protocol
SFT	Validation-loss filter, then procedural holdout success. Phase A LR sweep at rank 64: 1.5×10^{-4} , 5×10^{-4} , 1.5×10^{-3} ; Phase B rank sweep: 4, 8, 16, 32, 64. Selected LR= 1.5×10^{-4} , rank 8; confirmed across three seeds ($76.1\% \pm 0.8$ pp at epoch 10).
RL	Final <i>got_root</i> after 200 procedural training steps. LR sweep: 3.75×10^{-6} , 7.5×10^{-6} , 1.5×10^{-5} , 3×10^{-5} , 6×10^{-5} . Selected LR= 1.5×10^{-5} from the selected SFT checkpoint, keeping rank and α .

TABLE 6. HYPERPARAMETERS FOR BOTH TRAINING STAGES.

	SFT	RL
Learning rate	1.5×10^{-4}	1.5×10^{-5}
LR scheduler	linear (no warmup)	constant
Optimizer	AdamW 8-bit ($\beta_2=0.95$)	AdamW
LoRA rank / alpha	8 / 32	8 / 32
LoRA targets	all linear + <code>lm_head</code> [†]	
LoRA dropout	0.0	0.0
Precision	QLoRA (4-bit)	bf16
Sequence length	32,768	32,768
Batch size	8	80 (rollouts: 8)
Checkpoint / steps	epoch 10 of 10	step 300 of 1,000
Training horizon (rounds)	—	20
Hardware	4×H100, ≈1 hr 41 min	4×H100, ≈11.5 hr

[†]`q, k, v, o_proj, gate, up, down_proj, lm_head`

`--privileged ... /bin/bash -p` variants and 384 `test_credentials` attempts, treating root inside throw-away containers as success. It never attempts the host-bind-mount (`-v /:/host`) that would actually yield host root, and the run exhausts the 60-round budget without escalating.

TABLE 7. EVALUATION INFERENCE SETTINGS PER MODEL GROUP. API MODELS ACCESSED VIA OPENROUTER [44].

	Qwen3 family local	Gemma local	DeepSeek API	Claude API
Weight format	bf16 [§]	4-bit BnB	—	—
Temperature	0.7	0.7	default [†]	default [‡]
top-p	0.8	0.95	default [†]	default [‡]
top-k	20	—	—	—
Max output	2,048	2,048	2,048	2,048
Context cap	32K	32K [¶]	128K	200K
Reasoning	—	thinking	high	high
Parser	Hermes	Gemma4	—	—

[§]Qwen SFT used 4-bit QLoRA; static inference used bf16 vLLM weights.

[¶]Gemma local serving used unsloth/gemma-4-31B-it-unsloth-bnb-4bit; model-card maximum is 256K, capped to 32K to fit an RTX 5090-class memory budget.

[†] $T=1.0$, top-p=0.95. [‡] $T=1.0$, top-p=1.0.

TABLE 8. TEACHER REQUEST-LEVEL SETTINGS. *DeepSeek V4 Flash* (`DEEPSEEK/DEEPSEEK-V4-FLASH`) IS ACCESSED THROUGH OPENROUTER [44] AND PINNED TO DEEPSEEK PROVIDER.

	Trace (guided) rollout	Trace (unguided) rollout	Short-reason. rewrite
Temperature	default [†]	default [†]	0
top-p	default [†]	default [†]	default [†]
Max output	default [†]	default [†]	default [†]
Thinking	disabled	enabled	disabled
Trace seq. len. cap	32,768	32,768	—

[†]Not set on the request; uses DeepSeek provider defaults at trace-collection time.

C Prompt Templates

All prompt templates are released in the artifact repository¹ under `src/prompts/`: `privilege_escalation.jinja` is the deployment system prompt shared by SFT, RL, and static-benchmark evaluation; `trace_collection.jinja` adds hidden solution guidance only for the

TABLE 9. TRACE-DESIGN ABLATION ON PROCEDURAL HOLDOUT ENVIRONMENTS. EACH ROW TRAINS *Qwen3 4B* WITH THE SAME SFT HYPERPARAMETERS ON 2,000 TRACES AND EVALUATES 500 HOLDOUT RUNS. SUCCESS CELLS SHOW PERCENT (SUCCESSSES/500); WILSON INTERVALS ARE RUN-LEVEL AND EXCLUDE TRAINING-SEED VARIATION. THE STATIC BENCHMARK IS NOT USED FOR THIS CHOICE.

Collection	Trace format	Avg. tokens/trace	Train >16k	Success $r \leq 20$	95% CI	Success $r \leq 60$
Guided	No reasoning	3,122	0	51.4% (257/500)	47.0–55.8	52.0% (260/500)
Guided	Short reasoning	3,213	0	60.2% (301/500)	55.8–64.4	60.4% (302/500)
Guided	Long reasoning	3,525	0	65.2% (326/500)	60.9–69.2	66.4% (332/500)
Unguided	No reasoning	7,314	119	56.4% (282/500)	52.0–60.7	61.0% (305/500)
Unguided	Short reasoning	7,488	126	58.2% (291/500)	53.8–62.4	58.4% (292/500)
Unguided	Long reasoning	8,751	196	73.2% (366/500)	69.2–76.9	74.8% (374/500)

TABLE 10. COMPLEMENTARY REWARD-DESIGN DETAILS AT $r \leq 20$. THE MAIN TEXT USES FIGURES 3 AND 4; THIS TABLE GIVES WILSON 95% CIS AND FINAL-CHECKPOINT STABILITY VALUES FOR THE SAME VALID PROCEDURAL HOLDOUT RUNS.

Reward	Peak	95% CI	Final
SFT init.	73.2%	69.2–76.9	—
OUTCOME	88.0%	80.2–93.0	83.0%
+ROUND	90.0%	82.6–94.5	75.0%
+COST*	90.0%	82.6–94.5	87.0%
+ROUND+COST	87.0%	79.0–92.2	79.0%

guided control and strips it before dataset assembly; `short_reasoning_rewrite.jinja` derives the short-reasoning SFT variant; and `privilege_escalation_budget.jinja` and `privilege_escalation_minimal.jinja` are the two prompt-sensitivity variants evaluated below. The shared deployment-time user instruction and the no-tool-call recovery nudge are held fixed across all variants and are released alongside the templates.

Prompt-Sensitivity Audit

Only the system prompt varies across rows of Table 11; tool schemas, the deployment user instruction, the no-tool nudge, and the sampling parameters are held fixed. *detailed* is the canonical deployment prompt used throughout the rest of the paper; *budget* adds an explicit efficiency objective and a short pre-tool reasoning checklist; *minimal* strips procedural guidance to just the task statement, access constraints, and completion condition.

TABLE 11. POST-HOC PROMPT-SENSITIVITY AUDIT ON THE STATIC BENCHMARK. VALUES ARE SUCCESS PERCENTAGES OVER $N=240$ VALID RUNS PER SYSTEM-PROMPT CELL. SPREAD REPORTS THE PER-SYSTEM PROMPT SPREAD AT EACH BUDGET.

System	$r \leq 20$				$r \leq 60$			
	detailed	budget	minimal	spread	detailed	budget	minimal	spread
<i>Qwen3 4B</i>	42.1	25.4	21.7	20.4	47.5	35.0	31.2	16.2
<i>Qwen3 4B SFT</i>	80.8	76.2	75.8	5.0	82.5	79.2	80.0	3.3
<i>PrivEsc-LLM 4B</i>	90.0	91.7	88.3	3.3	90.0	92.1	91.7	2.1
<i>Gemma 4 31B</i>	80.4	82.5	77.1	5.4	89.2	86.7	83.8	5.4

LLM Usage Statement

LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality. Generative AI tools were used to refine prose, improve clarity, and assist with code formatting in figures and listings. All technical contributions, experimental design, methodology, results, and analysis are the authors’ own. The authors take full responsibility for the content of the paper and all reported results.

LLMs also feature in the technical contribution itself: the system under study is an LLM agent for Linux privilege escalation. LLM-based teachers were used to collect supervised demonstration traces in procedurally generated environments (Section 3.3), and the resulting models were then trained, evaluated, and analyzed. All such uses, including model identifiers, prompts, sampling parameters, and accounting of inference cost, are documented in the methodology and appendices.